C++ | CPl

Summary

CONTENTS

1. Why C++? 4
2. Intro 4
   - 2.1. Compilation 5
   - 2.2. Declarations and definitions 5
     - 2.2.1. `#include` guards 5
     - 2.2.2. Functions 5
     - 2.2.3. Values and references 6
     - 2.2.4. Alias Declarations 6
   - 2.3. Libraries 6
3. Values and Streams 6
   - 3.1. Variables 6
   - 3.2. Literal Values 7
   - 3.3. Expressions 8
     - 3.3.1. Type Conversion 8
     - 3.3.2. Assignment Operation 8
     - 3.3.3. Logic Operations 8
     - 3.3.4. Floating Point Numbers (IEEE754) 8
     - 3.3.5. Strings 8
   - 3.4. Basic Streams 9
     - 3.4.1. Errors 9
     - 3.4.2. `std::string` 10
     - 3.4.3. `int` 10
     - 3.4.4. Boolean Conversion 10
     - 3.4.5. IO Headers 11
4. Sequences and Iterators 11
   - 4.1. `std::array<T, N>` 11
   - 4.2. `std::vector<T>` 12
   - 4.3. Iteration 13
     - 4.3.1. Algorithms 13
     - 4.3.2. Ranges 14
     - 4.3.3. Putting it all together 14
   - 4.4. Mutating 14
     - 4.4.1. Filling 15
   - 4.5. Finding and counting elements 15
   - 4.6. I/O 15
5. Functions 16
   - 5.1. Basics 16
     - 5.1.1. Arguments 16
     - 5.1.2. Return types 16
     - 5.1.3. Overloading 17
     - 5.1.4. Default Arguments 17

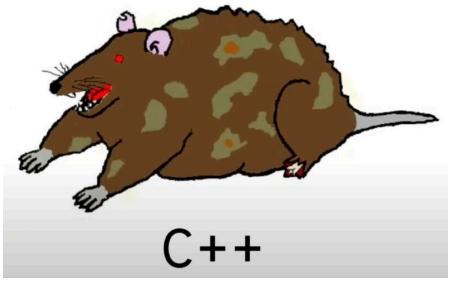
5.1.5. Functions as Parameters ... 18
5.1.6. Lambdas ... 18
5.2. Failing ... 20
5.2.1. Exceptions ... 20
6. Classes and Operators ... 22
6.1. Declaration ... 22
6.1.1. Access specifiers ... 23
6.1.2. Member variables ... 23
6.1.3. Constructor ... 23
6.1.4. Destructor ... 24
6.2. Inheritance ... 24
6.3. Implementation ... 25
6.3.1. Constructors ... 25
6.3.2. Member functions ... 26
6.3.3. Static member variables ... 27
6.4. Operator overloading ... 28
6.4.1. Defaulting comparison Operators ... 28
6.4.2. Ordering ... 29
6.4.3. `std::ostream` `std::istream` ... 29
7. Namespaces ... 30
7.1. Name resolution ... 31
7.1.1. Using ... 31
7.1.2. Argument Dependent Lookup (ADL) ... 31
8. Enums ... 32
8.1. Defining ... 32
8.2. Scoped vs unscoped ... 32
8.3. Implementing ... 33
8.3.1. Specifying the underlying type ... 33
9. STL Containers and Iterators ... 34
9.1. Sequence Containers ... 35
9.1.1. `std::array` ... 35
9.1.2. `std::deque<T>` ... 35
9.1.3. `std::list<T>` ... 35
9.1.4. `std::forward_list<T>` ... 35
9.2. Adapters ... 36
9.2.1. `std::stack` ... 36
9.2.2. `std::queue` ... 36
9.2.3. `std::priority_queue` ... 36
9.3. Associative Containers ... 36
9.3.1. `std::set<T>` ... 37
9.3.2. `std::map<K, V>` ... 37
9.3.3. `std::multiset<T>` and `std::multimap<K, V>` ... 37
9.4. Hashed Containers ... 38
9.4.1. `std::unordered_set<T>` ... 38
9.4.2. `std::unordered_map<K, V>` ... 38
9.5. Iterators ... 38
9.5.1. Categories ... 38
9.5.2. Functions ... 40

9.5.3. A word on mutability 41
Bibliography 42

WARN:
This document is still under development and will be completed around 1.2.2027

1. WHY C++?

– Multi-paradigm language, now you have a thousand and one way to shoot yourself in your foot and obliterate your PC
– Being called stupid when you think that the `move` function actually moves things or if you confuse a const pointer with a pointer of const
– ISO standard: C++23. The higher the number the hotter the mess of poorly designed choices that have accumulated over decades into a chaotic pile of obsoleteness
– Good excuse for alcoholism
– Zero-cost abstractions (if you like learning the whole stdlib by heart and don't mind the cost of debugging template errors for hours on end)
– Headaches (if you're a masochist)
–
– It's a never ending journey. Even if you dedicate your whole life to learning C++, you'll never reach the day where you'll know all of the stdlib.
– "Undefined behavior" is defined (in the standard)
– Ever wanted to have 100 different ways to do the same thing? Now you can
– Has a pretty mascot [1]



2. INTRO

– `main()` implicitly returns 0
– Return types are written in front of the function name or as trailing return-types in declarations (because who needs consistency, amirite)

Nice and clean: `int main() { }` Insanity: `auto main() → int { }`

– Header files can be either `*.hpp` or `*.h`
– Regarding include order: It is good practice to include own-headers ("") before system-headers (<>). The first is usually used for the header files belonging to the project itself, while the latter is used for includes of the standard library and external libraries. However, ***this difference is only conventional, as any toolchain could just specify their own lookup rules for the two distinct kinds of includes***.

Do's	*Don'ts*
	Using global variables
	Redundant returns in `main()` that would be implicit
	`using namespace`
	Not initializing a variable upon definition using {}
	Not using `const` enough
	Using `sdt::cin` and `sdt::cout`
	Not checking `stream.good()`

Do's	*Don'ts*
	Not removing stream fail flag with `stream.clear()`
	Not using `#include` guards
	Using plain C-Arrays
	Using plain `for(int i; ...)` loops
	Returning from a function by const value (unnecessary and stupid)
	Returning reference to a local variable
	Throwing primitives. Don't ask why exactly should you be able to do this in the first place.
	Making class member variables `const`
	Adding member variables to communicate between member function calls
	Throwing an exception in a destructor
	Type-conversion constructors not being declared explicit
	TODO: make this column real big for the lulz

2.1. COMPILATION

3 Phases of compilation:

Preprocessor: Textual replacement of preprocessor directives (`#include`)

Compiler: Translation of C++ code into machine code (source file to object file)

Linker: Combination of object files and libraries into libraries and executables

2.2. DECLARATIONS AND DEFINITIONS

All things with a name that you use in a C++ program must be declared before you can do so
– E.g. a function that you call (major difference from C)
– A type that you use for a variable (except some built-ins)
– A variable that you use

There can only be one definition of the same function (***One Definition Rule***/ODR), in contrast to declaration, which can be done several times.

2.2.1. #include guards

`#include` guards ensure that a header file is only included once. Necessary for functions and class type definitions, good practice either way.

```
#ifndef GUARD_HPP_
#define GUARD_HPP_
// ...
#endif /* GUARD_HPP_ */
```

2.2.2. Functions

Declarations

```
auto          <function-name>(<parameters>) -> <return-type>;
<return-type> <function-name>(<parameters>);
```

Definitions

```
auto          <function-name>(<parameters>) → <return-type> { /*body*/}
<return-type> <function-name>(<parameters>)                 { /*body*/}
```

If a function has a non-void return type it must return a value on every path (or throw an exception).

2.2.3. Values and references

```
struct Point {
  int x;
  int y;
};
```

Copying vs. Sharing:

```
Point point{1, 20};

Point copiedPoint{point};
Point & sharedPoint{point};

auto copydPointParam(Point point) → Point { }
auto referencePointParam(Point & point) → Point { }
```

2.2.4. Alias Declarations

A type alias can help to abbreviate type names.

```
using <alias-name> = <type>;
```

***Example** 1:*

```
using input = std::istream_iterator<std::string>;
input eof{};
input in{std::cin};
```

2.3. LIBRARIES

– catch2
– boost

TODO:
cmake

3. VALUES AND STREAMS

3.1. VARIABLES

```
<type> <variable-name>{<initial-value>};
```

Using = or {} for initialization with a value supplied we can have the compiler determine its type.

```
auto const i = 5;
```

Initialization might be omitted but that is bad practice and potentially dangerous.

```
double x;
```

Adding the const keyword in front of the name makes the variable a single-assignment variable, aka a constant which is immutable and must be initialized. Some constants are required to be fixed at compile time (`constexpr`). The keyword `const` also appears in other contexts.

```
int const theAnswer{42};
double constexpr pi{3.14};
```

You should use `const` whenever possible for non-member variables.

3.2. LITERAL VALUES

Literal Example	*Type*	*Value*
`'a'`	char	Letter a, value: 97
`'\n'`	char	<NL> character, value: 10
`'\x0a'`	char	<NL> character, value: 10
`1`	int	1
`42L`	long	42
`5LL`	long long	5
`int{} // (not really a literal)`	int	0 (default value)
`1u`	unsigned int	1
`42ul`	unsigned long	42
`5ull`	unsigned long long	5
`020`	int	16 (octal 20)
`0x1f`	int	31 (hex 1F)
`0XFULL`	unsigned long long	15 (hex F)
`0.f`	float	0
`.33`	double	0.33
`1e9`	double	1000000000 (109)
`42.E-12L`	long double	0.00000000042 (42*10-12)
`.3l`	long double	0.3
`"hello"`	char const [6]	Array of 6 chars: h e l l o <NUL>
`"\012\n\\"`	char const [4]	Array of 4 chars: <NL> <NL> <NUL>

3.3. EXPRESSIONS

	Arithmetic	*Bit-operators*	*Logic*
Unary	`+ - ++ --`	`~`	`!`
Binary	`+ - * / %`	`& | ^ << >>`	`&& || < > ⩽ ⩾ == ≠`
Tertiary			`? :`

– Fraction results of integer operations are always rounded down
– Operator precedence

3.3.1. Type Conversion

– Integer to boolean conversion `0` ⟶ `false` / every other value ⟶ `true`
– Automatic type conversion if values of different types are combined in an expression, **unless in braced initialization**
– Dividing integers by zero is **undefined behavior**

3.3.2. Assignment Operation

```
// vvvvv lvalue
   myVar = 6 * 7;
//         ^^^^^ rvalue
```

3.3.3. Logic Operations

Logical operators and conditional statements are generous to accept numeric values as statement of truth

```
//    v returns boolean
if (a < b < c);
//        ^ coalesces to 0 if false, 1 if true
```

3.3.4. Floating Point Numbers (IEEE754)

– Use `double` – usually most efficient on current hardware and default for floating point literals.
– Use float only if memory consumption is utmost priority and precision and range can be traded (on 64bit often not beneficial).
– Remember there are legal double values that are not numbers: `NaN, +Inf, -Inf`.
– Comparing floating points for equality (==) is usually wrong
  – In Catch2 you have the option to specify a matcher that checks in a relative range to the expected value `REQUIRE_THAT(actual, Catch::Matchers::WithinRel(expected, delta));`

3.3.5. Strings

`std::string` is C++'s type for representing sequences of char (which is often only 8 bit). For working with non-ASCII characters an external library is advised. It is mutable and iterable with iterators.

```
#include <string>
std::string statement{"Rust ftw"};
```

String literals like `"ab"` are not of type `std::string`, they're a null-terminated array of const characters (`char const[3]`). But `"ab"s` is an `std::string` but requires `using namespace std::literals`.

3.4. BASIC STREAMS

Streams aren't values, because they cannot be copied. So functions taking a stream object must take it as a reference.

Pre-defined globals: `std::cin` `std::cout`, should only be used in `main()`. "shift" operators read into variables or write values and can be chained:

```
std::cin >> x;
std::cout << "the value is " << x << '\n';
```

Output can be formatted using I/O manipulators .

***Example** 2:*

```
#include <iostream>
#include <iomanip>
#include <ios>

auto main() -> int {
  std::cout << 42 << '\t'
    << std::oct << 42 << '\t'
    << std::hex << 42 << '\n';
  std::cout << 42 << '\t' // std::hex is sticky
    << std::dec << 42 << '\n';
  std::cout << std::setw(10) << 42
    << std::left << std::setw(5) << 43 << "*\n";
  std::cout << std::setw(10) << "hallo" << "*\n";
  double const pi{std::acos(0.5) * 3};
  std::cout << std::setprecision(4) << pi << '\n';
  std::cout << std::scientific << pi <<  '\n';
  std::cout << std::fixed << pi * 1e6 <<  '\n';
}
```

3.4.1. Errors

Streams have a state that denotes if I/O was successful or not. If a previous read already failed, subsequent reads fail as well.

State Bit Set	*Query*	*Entered*
`<none>`	`is.good()`	`initial` `is.clear()`
`failbit`	`is.fail()`	formatted input failed
`eofbit`	`is.eof()`	trying to read at end of input
`badbit`	`is.bad()`	unrecoverable I/O error

– Formatted input on stream `is` must be checked for `is.fail()` and `is.bad()`
– If failed, `is.clear()` the stream and consume invalid input characters before continuing

3.4.2. `std::string`

Reading a `std::string` can not go wrong, unless the stream is already `!good()`
– The content of the `std::string` is replaced
– Maybe the `std::string` is empty after reading

3.4.3. `int`

Reading an `int` results in:
– No error recovery
– One wrong input puts the stream into status fail
– Characters remain in input

Example 3: **Robust way of reading an `int`**

```
auto readInt(std::istream & in) -> int {
  std::string line{};
  while (getline(in, line)) {
    std::istringstream is{line};
    int res{-1};
    if (is >> res)
      return res;
  }
  return -1;
}
```

Example 4: **Reading an `int` and skipping invalid characters**

```
auto readInt(std::istream & in) -> int {
  while (in.good()) {
    int res{-1};
    if (in >> res)
      return res;
    in.clear();
    in.ignore();
    // alt: in.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    // ignores whole line
  }
  return -1;
}
```

3.4.4. Boolean Conversion

Result of `is >> res` is the `std::istream` object itself. The stream object converts to `bool` (in if and loop conditions):
– `true` if the last reading operation has been successful
– `false` if the last reading operation failed somehow (formatting, stream end or another problem)

`good()` for pre-checking “can I still read?” and `bool` conversion for post-checking “did I read correctly?”.

ios_base::iostate flags			basic_ios accessors					
eofbit	failbit	badbit	good()	fail()	bad()	eof()	operator bool	operator !
false	false	false	true	false	false	false	true	false
false	false	true	false	true	true	false	false	true
false	true	false	false	true	false	false	false	true
false	true	true	false	true	true	false	false	true
true	false	false	false	false	false	true	true	false
true	false	true	false	true	true	true	false	true
true	true	false	false	true	false	true	false	true
true	true	true	false	true	true	true	false	true

3.4.5. IO Headers

iosfwd: contains only the declarations for `std::ostream` and `std::istream`.
In header files (`.hpp`) this is usually sufficient when the streams are only used in function declarations

istream and ostream: contain the implementation of the corresponding stream and operators.
Usually, these are required in source files (`.cpp`) when the streams are actually used in functions

iostream: contains all of the above and additionally `std::cout`, `std::cin`, `std::cerr`.
This is only required in the source file containing the `main()` function, because only there the global standard IO variables shall be used

4. SEQUENCES AND ITERATORS

4.1. STD::ARRAY<T, N>

```cpp
#include <array>

std::array<int, 6> emptyArray{};
```

Is a fixed-size Container

– T is a template type parameter (= placeholder for type)

– N is a positive integer, template non-type parameter (= placeholder for a value)

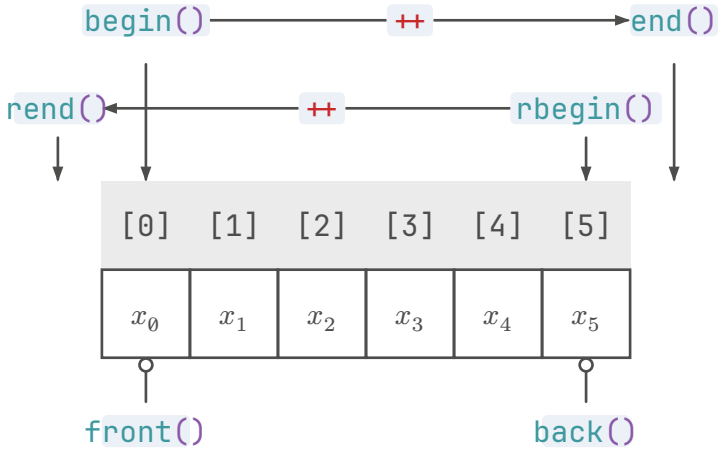
It can be initialized with a list of elements

– The size of an array must be known at compile-time and cannot be changed

– Otherwise, it contains N default-constructed elements

The size is bound to the array object and can be queried using `.size()`

Element access using subscript operator `[]` or `.at()`
– `.at()` throws `std::out_of_range exception` on invalid index access
– `[]` has undefined behavior on invalid index access



4.2. `std::vector<T>`

```
#include <vector>

std::vector<int> numbers{1, 2, 3, 4, 5};
```

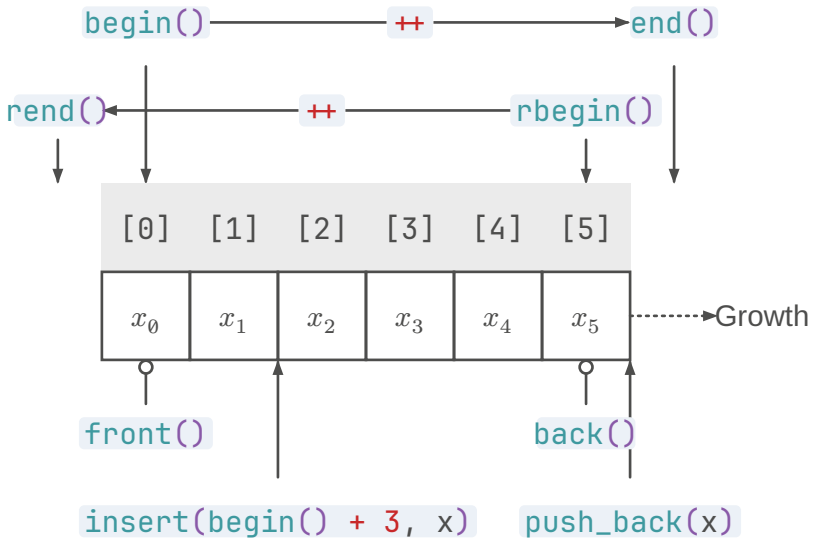
Is a Container = contains its elements of type T (no need to allocate them). Works similar to an `java.util.ArrayList<T>` in java, although it doesn't store the Elements as references but as copies (use `std::vector<&T>` for that purpose).

`std::vector` can be initialized with a list of elements
– The list can be empty: `std::vector<double> vd{};`
– Other construction means might need parentheses (legacy)

When an initializer is given, the element type can be deduced! `std::vector{1, 2, 3, 4, 5};`

Parenthesis at definition allow providing initial size, when type of elements is not numeric `std::vector<std::string> words{6}`

Index variable type is "unsigned" `std::size_t` or `std::vector<T>::size_type`



4.3. ITERATION

	const	*non-const*
reference	`for (auto const & cref : v) { }`	`for (auto & ref : v) { }`
copy	`for (auto const ccopy : v) { }`	`for (auto copy : v) { }`

Only use this if the iterator is required in the loop:

```
for (auto it = std::begin(v); it != std::end(v); ++it) {
  std::cout << (*it)++ << ", ";
}
```

Guarantee to just have read-only access: `std::cbegin()` and `std::cend()`

4.3.1. Algorithms

```
#include <algorithm>
```

Each algorithm takes iterator arguments: The range(s) of elements to apply an algorithm to is specified by iterators.

```
auto count_blanks(std::string s) -> size_t {
  return std::count(s.cbegin(), s.cend(), ' ');
}
```

Example 5: **Sum**

```
#include <numeric>
std::vector<int> v{5, 4, 3, 2, 1};
std::cout << std::accumulate(std::cbegin(v), std::cend(v), 0)<< "=sum\n";
```

Example 6: **Nr. of elements**

```
#include <iterator>
void printDistanceAndLength(std::string s) {
  std::cout << "distance: "<< std::distance(s.begin(), s.end()) <<'\n';
  std::cout << "in a string of length: "<< s.size()<<'\n';
}
```

Example 7: **Foreach**

```
auto print(int x) -> void {
  std::cout << "print: "<< x << '\n';
```

```
}
auto printAll(std::vector<int> v) → void {
  std::for_each(std::crbegin(v), std::crend(v), print);
}
```

Example 8: **Foreach with lambdas**

```
auto printAll(std::vector<int> v, std::ostream & out) → void {
  std::for_each(std::cbegin(v), std::cend(v), [&out](auto x) {
    out << "print: "<< x << '\n';
  });
}
```

4.3.2. Ranges

```
auto printAll(std::vector<int> v, std::ostream & out) → void {
  std::ranges::for_each(v, [&out](auto x) {
    out << "print: "<< x << '\n';
  });
}
```

4.3.3. Putting it all together

Iterators connect containers and algorithms.

Containers	*Iterators*	*Algorithms*

`std::vector<T>`
`std::string`
`std::set<T>`
`std::map<K, V>`
`...`
	`std::begin()`
	`std::end()`
	`std::rbegin()`
	`std::rend()`
	`...`
		`std::count(b, e, val)`
		`std::ranges::count(r, val)`
		`std::find(b, e, val)`
		`std::accumulate(b, e, start)`
		`std::copy(b, e, b_target)`
		`...`

4.4. MUTATING

Append: `v.push_back(<value>);`

Insert anywhere: `v.insert(<iterator-position>, <value>);`

When using the `std::copy` algorithm the target has to be an iterator too

```
std::copy(<in-begin-iterator>, <in-end-iterator>, <out-begin-iterator>);
std::ranges::copy(<in-range>, <out-begin-iterator>);
```

To append elements to another list, use `std::back_inserter()`:

```
std::vector<int> source{1, 2, 3}, target{};
std::copy(source.cbegin(), source.cend(), std::back_inserter(target));
std::ranges::copy(source, std::back_inserter(target));
```

4.4.1. Filling

Filling a vector with the same values can be done in multiple ways:

```
// std::fill requires a vector with existing elements to be overwritten
std::vector<int> v{};
v.resize(10);
std::fill(std::begin(v), std::end(v), 2);
std::ranges::fill(v, 2)
// or
std::vector<int> v(10);
std::fill(std::begin(v), std::end(v), 2);
std::ranges::fill(v, 2)
// or
std::vector<int> v(10, 2);
```

Filling a vector with different values can also be done in multiple ways (wowzers):

The algorithms `std::generate()` and `std::generate_n()` fill a range with computed values. Either use `std::back_inserter` or a non-empty container respectively. The `std::iota()` algorithm fills a range with subsequent values.

4.5. FINDING AND COUNTING ELEMENTS

`std::(ranges::)find()` and `std::(ranges::)find_if()` return an iterator to the first element that matches the value or condition. If no match exists the end of the range is returned.

Similarly `std::(ranges::)count()` and `std::(ranges::)count_if()` return the number of matching elements in a range.

4.6. I/O

Streams cannot be used with algorithms directly.

Containers	*Iterators*	*Algorithms*

`std::ostream`
`std::istream`
	`std::ostream_iterator()`
	`std::istream_iterator()`
		`std::count(b, e, val)`
		`std::ranges::count(r, val)`
		`std::copy(b, e, b_target)`
		`...`

`std::istream_iterator<T>` reads values of type T from the given `std::istream`. End iterator is the default constructed `std::istream_iterator<T>{}`, it ends when the stream is no longer `good()`.

`std::ranges::istream_view<T>` combines `in` and `eof`.

`std::istream_iterator<T>` uses operator `>>` for input, which skips white space. For an exact copy, use `std::istreambuf_iterator<char>`, which uses `std::istream::get()` to get every character.

This only works with char-like types. `std::noskipws` stream manipulator can also be used for this purpose.

***Example** 9:*

```
using input = std::istreambuf_iterator<char>;
input eof{};
input in{std::cin};
std::ostream_iterator<char> out{std::cout, " "};
std::copy(in, eof, out);
```

To fill a vector from a stream you can either use copy with `std::back_inserter(v)` (It uses `v.push_back()` internally)

```
using input = std::ranges::istream_view<int>;
std::vector<int> v{};
std::ranges::copy(input{std::cin}, std::back_inserter(v));
```

or construct the `std::vector<T>` directly from two iterators

```
using input = std::istream_iterator<int>;
input eof{};
std::vector<int> const v{input{std::cin}, eof};
```

5. FUNCTIONS

5.1. BASICS

5.1.1. Arguments

	const	*non-const*
reference	`auto f(char const & c) -> void { }`	`auto f(char & c) -> void { }`
copy	`auto f(char const c) -> void { }`	`auto f(char c) -> void { }`

5.1.2. Return types

In function definitions the trailing return-type could be omitted. The actual return type will be deduced from the return statements in the function's body.

	const	*non-const*
reference	`auto f() -> type const & { }`	`auto f() -> type & { }`
value	`auto f() -> type const { }`	`auto f() -> type { }`

Example 10: **Incorrect**

```
#include <iostream>

// here the compiler can't deduce the return type!
auto maxValue(int f, int s, int t);

// this won't compile
int main() {
  std::cout << maxValue(1, 2, 3);
}
```

5.1.2.1. Const reference

`const &` extends the life-time of the temporary object, until the end of the block.

Example 11: **The POI will be copied into the returned `std::vector` object**

```
auto createPOI(Coordinate) -> POI;
auto allPOIs(Coordinate const location) -> std::vector<POI> {
  POI const & migros = createPOI(location);
  return std::vector{migros};
}
```

5.1.3. Overloading

```
auto incr(int & var) -> void;
auto incr(int & var, unsigned delta) -> void;
```

The same function name can be used for different functions if parameter number or types differ
– Functions cannot be overloaded just by their return type
– If the parameter type is only different in reference/object there will be ambiguities
– If the parameter is only different in type, there might be ambiguities (e.g. passing a `long` into `int` vs. `double`)

Resolution of overloads happens at compile-time (Ad hoc polymorphism). That also means that the internal name of a function also contains its parameter types as significant information.

5.1.4. Default Arguments

```
auto incr(int & var, unsigned delta = 1) -> void;
```

A function declaration can provide default arguments for its parameters from the right. Definition doesn't need to/shouldn't repeat:

```
auto incr(int & var, unsigned delta) -> void {
  var += delta;
}
```

If n default arguments are provided, the behavior is, as if $n + 1$ versions of the function were declared.

5.1.5. Functions as Parameters

```
auto applyAndPrint(double x, auto f(double) → double) → void {
  std::cout << "f(" << x << ") = " << f(x) << '\n';
}
```

Type signatures (legacy):

```
auto f(double) → int
int f(double)
auto (&ref)(double) → int
int (&ref)(double)
```

Drawback: A function parameter declared in this way does not accept a lambda with a capture. Use `std::function`.

5.1.5.1. `std::function`

```
#include <functional>
```

Modern C++ approach: `std::function` template, which also allows passing lambdas (with capture)

```
std::function<auto(double) → int>
std::function<int(double)>
```

***Example** 12:*

```
auto applyAndPrint(double x, std::function<auto(double) → double> f) →
void {
  std::cout << "f(" << x << ") = " << f(x) << '\n';
}
auto main() → int {
  double factor{3.0};
  auto const multiply = [factor](double value) {
    return factor * value;
  };
  applyAndPrint(1.5, multiply);
}
```

5.1.6. Lambdas

```
[<capture>](<parameters>) → <return-type> {
  <statements>
}
```

Capture names variables taken from the surrounding scope, or define new ones (= copy, & reference, rename possible, type deduced). The return type can be omitted if `void` or if inferrable for the compiler. Parameters can be `auto` if inferrable from context.

```
auto g = [](char c) → char {
  return std::toupper(c);
```

```
};
g('a');
```

Captured local copies are immutable, unless lambda is declared mutable and lives as long as the lambda lives.

```
int x = 5;
auto l = [x]() mutable {
  std::cout << ++x;
};
```

Capturing a local variable by reference requires the referenced variable to live at least as long as the lambda.

Capturing all (referenced) local variables by value:

```
int x = 5;
auto l = [=]() mutable {
  std::cout << ++x;
};
```

Capturing all (referenced) local variables by reference:

```
int x = 5;
auto const l = [&]() {
  std::cout << ++x;
};
```

Referenced variables will allow modification, unless it is originally declared const

Capturing `this` pointer allows accessing and modifying members of the class

```
struct S {
  auto foo() -> void {
    auto square = [this] {
      member *= 2;
    };
  }
private:
  int member{};
};
```

New local variable can be specified in capture
– New variable in capture has type `auto`
– Can be modified if lambda is mutable

```
auto squares = [x = 1]() mutable {
  std::cout << (x *= 2);
};
```

In captures multiple variables can be combined and separated with commas (`,`)

Within a single expression, such as a function call, sequence of evaluation is undefined! (except for the comma operator `,`)

5.2. FAILING

A function can fail if the ***precondition*** is violated (e.g. negative index, divisor is zero, …) or if the ***postcondition*** could not be satisfied (e.g. resources not available, cannot open file, …)

A function without preconditions has a so-called ***wide contract*** as opposed to ***narrow contract***.

What should you do, if a function cannot fulfill its purpose?

1) Ignore the error and provide potentially **undefined behavior**
   - Relies on the caller to satisfy all preconditions
   - Viable only if not dependent on other resources
   - Most efficient implementation (no unnecessary checks)
   - Simpler for the implementer but harder for the caller
2) Return a **standard result** to cover the error
   - Reliefs the caller from the need to care if it can continue with the default value
   - Can hide underlying problems
   - Often better if caller can specify its own default value
3) Return an **error code** or error value
   - Only feasible if result domain is smaller than return type
   - Burden on the caller to check the result
   - `std::string::npos, std::expected`
   - `std::optional` can be checked using `has_value()` or boolean conversion
4) Provide an **error status** as a side-effect
   - Requires reference parameter or global variable (bad)
   - Example: `std::istream`'s state (`good(), fail()`)
5) Throw an **exception**

5.2.1. Exceptions

```
throw value;
```

Any (copyable) type can be thrown and there are no means to specify what could be thrown. No meta-information is available as part of the exception. Exception thrown while exception is propagated results in program abort.

***Example** 13:*

```
throw std::invalid_argument{"reason"};
throw 15;
```

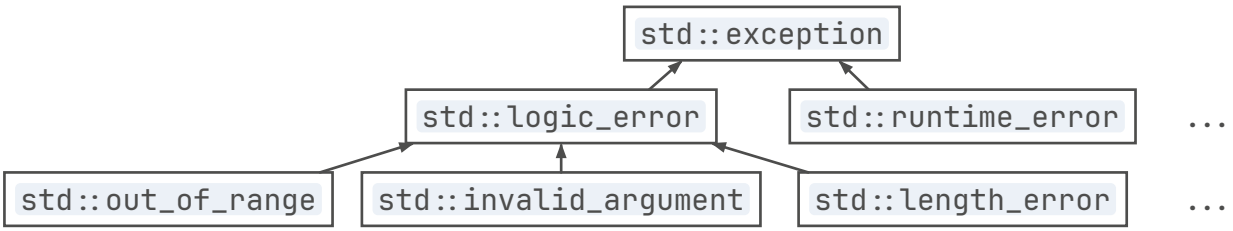
Catching can be done using a try catch block, where the first match wins. Throw by value, catch by const reference avoids unnecessary copying and allows dynamic polymorphism for class types.

Caught exceptions can be rethrown with `throw;`

***Example** 14:*

```
try {
  throwingCall();
} catch (type const & e) {
  //Handle type exception
} catch (type2 const & e) {
  //Handle type2 exception
} catch (...) {
  //Handle other exception types
}
```

The Standard Library has some pre-defined exception types that you can also use in `<stdexcept>`.



`std::exception` is the base class and provides the `what()` member function to obtain the “reason”, which is passed as a construction parameter.

Testing with Catch2 can be done as follows:

***Example** 15:* **REQUIRE_THROWS(code) when an exception is expected**

```
TEST_CASE("square_root of negative value throws") {
  REQUIRE_THROWS(square_root(-1.0));
}
```

***Example** 16:* **REQUIRE_THROWS_AS(code, exception_type) when a specific type is expected to be thrown**

```
TEST("at on empty vector throws std::out_of_range") {
  std::vector<int> empty_vector{};
  REQUIRE_THROWS_AS(empty_vector.at(0), std::out_of_range);
}
```

***Example** 17:* **REQUIRE_THROWS_WITH(code, string or string-matcher) for exception content**

```
TEST("parseInt of "one" throws with message") {
  REQUIRE_THROWS_WITH(parseInt("one"), "parse error - invalid digits in
'one'");
}
```

You can make your program terminate when an exception is thrown by using the `noexcept` keyword after a function param definition.

```
auto add(int lhs, int rhs) noexcept → int {
  return lhs + rhs;
}
```

TODO:
lifetime extension through const & can only be done for temporary lifetimes?

6. CLASSES AND OPERATORS

A class defines a new type and is usually defined in a header file

```
class <GoodClassName> {
  <member variables>
  <constructors>
  <member functions>
};
```

A good class:

– Does one thing well and is named after that
– Consists of member functions with only a few lines and no deep nesting
– Has a class invariant (Provides a guarantee about its state). Constructors establish that invariant
– Is easy to use without complicated protocol sequence requirements

6.1. DECLARATION

***Example** 18:* **Date.hpp**

```
#ifndef DATE_HPP_
#define DATE_HPP_
class Date {                                    // class keyword
  int year, month, day;                         // member variables
public:                                         // access specifier
  Date();                                       // default constructor
  Date(Date const &);                           // copy constructor
  Date(Date &&);                                // move constructor
  explicit Date(int year,                       // type conversion
                int month = 1,                  // constructor
                int day = 1);
  ~Date();                                      // destructor
  static auto isLeapYear(int year) → bool;      // public member function
private:                                        // access specifier
  auto isValidDate() const → bool;              // private member function
```

```
};                                                  // ends with semicolon ;
#endif
```

Keywords for defining a class are `class` and `struct`. Default visibility for members of the class are
– `private` for `class`
– `public` for `struct`

6.1.1. Access specifiers

Access specifiers (followed by a colon `:`)

private: visible only inside the class (and friends); for hidden data members

protected: also visible in subclasses

public: visible from everywhere; for the interface of the class

All subsequent members have this visibility. Each visibility can reoccur multiple times.

6.1.2. Member variables

Member variables have a type and a name `<type> <name>;` Don't make member variables `const` as it prevents copy assignment.

6.1.3. Constructor

A special member function with the name of the class and no return type. Can have an initializer list for member initialization.

```
<class-name>(<parameters>)
  : <initializer-list>
{}
```

6.1.3.1. Default constructor

```
Date d{};
```

It has no parameters, is implicitly available if there are no other declared constructors and has to initialize member variables with default values. E.g. `Date();`

6.1.3.2. Copy constructor

```
Date d2{d};
Date d3 = d;
```

It has one `Type const &` parameter and is implicitly available unless there is a move constructor or assignment operator. It copies all member variables and is usually not needed to be implemented explicitly.

6.1.3.3. Move constructor

```
Date d2{std::move(d)};
```

It has one `Type &&` parameter and is implicitly available unless there is a copy constructor or assignment operator. It moves all members and is usually not needed to be implemented explicitly.

6.1.3.4. Type conversion constructor

```
Date d{2001, 11, 9};
```

Constructors that are callable with a single argument of a different type are called type-conversion constructors.

They have either a single parameter or multiple parameter, which have default arguments for all parameters after the first.

Type-conversion constructors should be declared `explicit`, which avoids unexpected implicit conversion (the compiler is allowed to make one implicit conversion to resolve the parameters to a function).

6.1.3.5. Initializer list constructor

```
Container box{item1, item2, item3};
```

Has one `std::initializer_list` parameter and does not need to be explicit, as implicit conversion is usually desired.

TODO:
Initializer List constructors are preferred if a variable is initialized with {}

***Example** 19:*

```
struct Container {
  Container() = default;
  Container(std::initializer_list<Element> elements);
private:
  std::vector<Element> elements{};
};
```

6.1.4. Destructor

Named like the default constructor but with a leading ~. It must release all resources and must not throw an exception. It is implicitly available and called automatically at the end of the block for local instances.

6.2. INHERITANCE

Base classes are specified after the name (multiple inheritance is possible)

`class <name> : <base1>, ..., <baseN>`

Inheritance can specify a visibility (public, protected, private), which limits the maximum visibility of the inherited members. If no visibility is specified, the default of the inheriting class is used (class → private and struct → public)

***Example** 20:*

```
class Base {
private:
  int onlyInBase;
protected:
  int baseAndInSubclasses;
public:
```

```
class Sub : public Base {
  // Can see baseAndInSubclasses
  // and everyoneCanFiddleWithMe
};
```

```
  int everyoneCanFiddleWithMe
};
```

6.3. IMPLEMENTATION

6.3.1. Constructors

Example 21:

```
#include "Date.hpp"

Date::Date(int year, int month, int day)
  : year{year}, month{month}, day{day} {
  if (!isValidDate()) {                          // only create valid
    throw std::out_of_range{"invalid date"};     // instances, throw if
  }                                              // invalid
}
Date::Date() : Date{1980, 1, 1} {                // default values
}
Date::Date(Date const & other)
  : Date{other.year, other.month, other.day} {
}
```

One must initialize all members and enforce an invariant (properties for a value of the type that are always true).

6.3.1.1. Default Initialization

```
class <classname> {
  <type> <membername>{<default-value>};
};
```

Member variables can have a default value assigned (NSDMI - Non-Static Data Member Initializers). Such values are used if the member is not present in the initializer list of the constructor. The initializer list still overrides those values.

Example 22: **Default initialization for members**

```
// Date.hpp
class Date {
  int year{9999}, month{12}, day{31};
  Date();
  Date(int year, int month, int day);
};

// Date.cpp
```

```
Date::Date() { }                              // Date = 31.12.9999
Date::Date(int year, int month, int day)
  : year{year}, month{month}, day{day} { } // Date = day.month.year
```

6.3.1.2. Default constructors

Some special member functions are implicitly available in certain cases (e.g. default constructor is implicitly available if no other explicit constructor is declared).

(Re-)implementing the default behavior of the default constructor can be avoided by defaulting it `<constructor>() = default;`

The implicit constructor are not always wanted. One can make them explicitly private or delete them using `<constructor>() = delete;`

Possible for:
– Default constructor and destructor
– Copy/move constructor
– Copy/move assignment operator

6.3.1.3. Delegating constructors

Constructors can call other constructors. For that the constructor call has to be in the member initializer list.

***Example** 23:*

```
// Date.hpp
class Date {
  Date(Year year, Month month, Day day);
  Date(int year, int month, int day);
};

// Date.cpp
Date::Date(int year, int month, int day)
  : Date{Year(year), Month(month), Day(day)} { }
```

Similar are calls to constructors of base classes

```
Constructor(Parameters)
  : Base{Arguments} {
}
```

6.3.2. Member functions

An implicit `this` object is available in the scope of a member function. It's a pointer, so member variables can be accessed with an arrow: →.

If a member function is `const` then it must not modify members and can only call `const` member functions.

***Example** 24:*

```
auto Date::isValidDate() const -> bool {
  if (day ⩽ 0) // day is equivalent to this→day
    return false;
  switch (this→month) {
    case 1: case 3: case 5: case 7: case 8: case 10: case 12:
      return this→day ⩽ 31;
    case 4: case 6: case 9: case 11:
      return this→day ⩽ 30;
    case 2:
      return this→day ⩽ (isLeapYear(this→year) ? 29 : 28);
    default:
      return false;
  }
}
```

6.3.2.1. Static member functions

Static member functions cannot be `const` and have no `this` object. No `static` keyword is needed in implementation, only in declaration. Call with `<classname>::<member>()`

***Example** 25:*

```
class Date {
  static auto isLeapYear(int year) → bool;
};
```

6.3.3. Static member variables

No `static` keyword is needed in implementation, only in declaration. `static const` member can be initialized directly Access outside class with name qualifier: `<classname>::<member>`

***Example** 26:*

```
// Date.hpp
class Date {
  static const Date myBirthday;
  static Date favoriteStudentsBirthday;
  static const Date today{2018, 10, 16};
};

// Date.cpp
Date const Date::myBirthday{1964, 12, 24};
Date Date::favoriteStudentsBirthday{1995, 5, 10};
std::cout << Date::myBirthday;
```

6.4. OPERATOR OVERLOADING

Custom operators can be overloaded for user-defined types. They're declared like a function, with a special name. Unary operators have one parameter, binary operators have two (duh).

```
auto operator op(<parameters>) → <returntype>

<returntype> operator op(<parameters>)
```

Overloadable operators:

```
+    -    *   /   %   ^    &    |     ~      !        ,    =   <   >   ⩽   ⩾
++   --   <<  >>  ==  ≠    &&   ||    +=     -=       ⊭   %=  ^=  &=  ⊨   *=
⋘   ⋙   []  ()  →   →*  new  new[]  delete  delete[]  ⟺
```

Non-Overloadable operators:

```
::  .*  .  ?:
```

The three-way-comparison operator (also called spaceship operator, ⟺) can be implemented to provide all relational (`<`, `>`, ⩽, ⩾) comparisons at once.

***Example** 27:*

TODO:
fix typst cpp starship parsing

```cpp
#include <compare>

class Date {
  int year, month, day;
public:
  auto operator⩽>(Date const& right) const → std::strong_ordering {
    if (year ≠ right.year) {
      return year ⩽> right.year;
    }
    if (month ≠ right.month) {
      return month ⩽> right.month;
    }
    return day ⩽> right.day;
  }
  auto operator==(Date const& right) const → bool {
    return (*this ⩽> right) == std::strong_ordering::equal;
  } // Also allows ≠ comparison
};
```

6.4.1. Defaulting comparison Operators

The compiler can generate the three-way-comparison operator by defaulting it. Three-way compares the members in declaration order and implicitly generates the equality operator as well.

```
class Date {
  int year, month, day;
public:
  auto operator<=>(Date const& right) const = default;
};
```

If you only want equality/inequality, that is possible as well.

```
class Point {
  int x, y;
public:
  auto operator==(Point const& right) const = default;
};
```

6.4.2. Ordering

Strong order: Values that are equivalent/equal are indistinguishable. Either `a < b`, `a == b` or `a > b` must be `true`

`std::strong_ordering::less`	for `a < b`
`std::strong_ordering::equivalent`, `std::strong_ordering::equal`	for `a == b`
`std::strong_ordering::greater`	for `a > b`

Weak order: Values that are equivalent/equal ***may be*** distinguishable. Either `a < b`, `a == b` or `a > b` must be `true`. E.g. ignoring letter case on strings.

`std::weak_ordering::less`	for `a < b`
`std::weak_ordering::equivalent`	for `a == b`
`std::weak_ordering::greater`	for `a > b`

Partial order: Values that are equivalent/equal ***may be*** distinguishable. Either `a < b`, `a == b` or `a > b` ***can all be*** `false`. E.g. `double` as `NaN` always compares `false` even with itself! `std::isnan` is required.

`std::partial_ordering::less`	for `a < b`
`std::partial_ordering::equivalent`	for `a == b`
`std::partial_ordering::greater`	for `a > b`
`std::partial_ordering::unordered`	for none of the above

6.4.3. `std::ostream` `std::istream`

Output operator must be a free function.

– Parameters: `std::ostream&` and `Type const&`
– Returns `std::ostream&` for chaining output

***Example** 28:*

```
#include <ostream>

class Date {
  int year, month, day;
public:
  auto print(std::ostream& os) const -> void {
    os << year << "/" << month << "/" << day;
```

```
  }
};

inline auto operator<<(std::ostream& os, Date const& date) →
std::ostream& {
  date.print(os);
  return os;
}
```

Similar for `std::istream`: Input operator must be a free function.
– Parameters: `std::istream&` and `Type &`
– Returns `std::istream&` for chaining output

TODO:
exceptions come with runtime overhead due to compiler generated scaffolding and runtime type information

7. NAMESPACES

Namespaces are scopes for grouping and preventing name clashes. The global namespace has the `::` prefix, it can be omitted if unique. E.g. `::std::cout` is usually equal to `std::cout`. Namespaces can only be defined outside of classes and functions.

***Example** 29:*

```
namespace demo {
  auto foo() → void;              // 1
  namespace subdemo {
    auto foo() → void { }         // 2
  }
}
namespace demo {
  auto bar() → void {
    foo();                        // 1
    subdemo::foo();               // 2
  }
}
auto demo::foo() → void { }       // 1
auto main() → int {
  using demo::subdemo::foo;
  foo();                          // 2
  demo::foo();                    // 1
  demo::bar();
}
```

Omitting a name after `namespace` hides module internals, it should be only used in source files (`*.cpp`). The compiler chooses a unique internal identifier for the anonymous namespace.

7.1. NAME RESOLUTION

7.1.1. Using

– Import a name from a namespace into the current scope: `using std::string;`
– Alias: `using str = std::string;`
– Importing **all** namespaces: `using namespace std;`
  – use only in local scope to avoid pollution

7.1.2. Argument Dependent Lookup (ADL)

Argument Dependent Lookup is when the compiler encounters an unqualified function or operator call with an argument of a user-defined type and looks into the namespace in which that type is defined to resolve the function/operator.

E.g. it is not necessary to write `std::` in front of `for_each` when `std::vector::begin()` is an argument of the function.

7.1.2.1. Operators

Unqualified operator calls don't allow explicit namespace qualification. E.g.

`std::cout calendar::<< birthday;`

Functions and operators are looked up in the namespace of the type of their arguments first. E.g. namespaces `std` and `calendar` for `std::cout << birthday;`

***Example** 30:* **Cursed example of C++ implicitly doing shit nobody asked for**

```hpp
// Adl.hpp
namespace one {
  struct type_one{};
  auto f(type_one) -> bool {}
}
namespace two {
  struct type_two{};
  auto f(type_two) -> void {}
  auto g(one::type_one) -> void {}
  auto h(one::type_one) -> void {}
}
auto g(two::type_two) -> void {}
```

```cpp
// Adl.cpp
#include "Adl.hpp"
int main() {
  one::type_one t1{};
  f(t1);                // one::f
  two::type_two t2{};
  f(t2);                // two::f
  h(t1);                // ???
  two::g(t1);           // two::g
  g(t1);      // (error) ::g
  g(t2);                // ::g
}
```

7.1.2.2. More ADL Issues

Generic code (Templates) might not pick up a global `operator<<` in an algorithm call using `ostream_iterator` if the value output is from namespace `std` too. E.g. `std::vector<int>`

TODO:
W6 S17,18

8. ENUMS

```
enum <name> {
  <enumerators>
};
```

```
enum class <name> {
  <enumerators>
};
```

8.1. DEFINING

An enumeration creates a new type that can easily be converted to an integral type (unscoped enumeration only). Conversion from an integral type to an enumeration is not possible implicitly.

***Example** 31:*

```
enum DayOfWeek {
  Mon, Tue, Wed, Thu, Fri, Sat, Sun
};
```

The individual values (enumerators) are specified in the type. Unless specified explicitly, the values start with 0 and increase by 1. With = values can be specified for enumerators. Subsequent enumerators get the value incremented +1. Different enumerators can have the same value (because why shouldn't they).

***Example** 32:*

```
enum Month {
  jan = 1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec,
  january = jan, february, march, april, june, july, august,
  september, october, november, december, bestmonthoftheyear = 6
};
```

8.2. SCOPED VS UNSCOPED

Unscoped enumeration (without the `class` keyword) converts implicitly to `int`. Scoped enumeration on the other hand requires a `static_cast` like so:

```
int day = static_cast<int>(DayOfWeek::Sun);
```

Conversion from `int` to `enum` always requires a `static_cast`:

```
DayOfWeek tuesday = static_cast<DayOfWeek>(1);
```

Unscoped enumerators leak into surrounding scope, scoped enumerators don't:

***Example** 33:*

Unscoped

```
namespace calendar {
enum DayOfWeek {
  Mon, Tue, Wed, Thu, Fri, Sat, Sun
};
//Enumerators are visible here
}
auto isWeekend(calendar::DayOfWeek day) -> bool {
  return day == calendar::Sat ||
  day == calendar::Sun;
}
```

Scoped

```
namespace calendar {
enum class DayOfWeek {
  Mon, Tue, Wed, Thu, Fri, Sat, Sun
};
}
auto isWeekend(calendar::DayOfWeek day) -> bool {
  return day == calendar::DayOfWeek::Sat ||
  day == calendar::DayOfWeek::Sun;
}
```

8.3. IMPLEMENTING

Operators can be overloaded for enums just like with classes.

8.3.1. Specifying the underlying type

Enumerations can specify the underlying type by inheritance. The underlying type can be any integral type. This allows forward-declaring enumerations or it can be used to hide implementation details if defined as a class member.

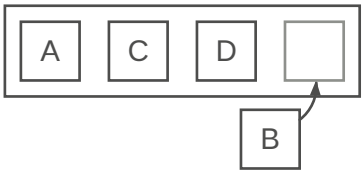
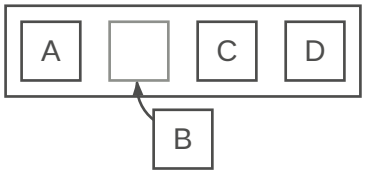
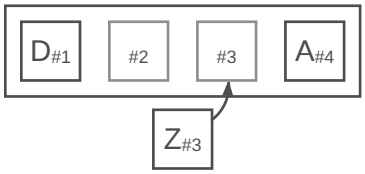
***Example** 34:*

```
enum class LaunchPolicy
  : unsigned char
{
  sync = 1,
  async = 2,
  gpu = 4,
  process = 8,
  none = 0,
}
```

TODO:
W6 S28,34(addable),35,36

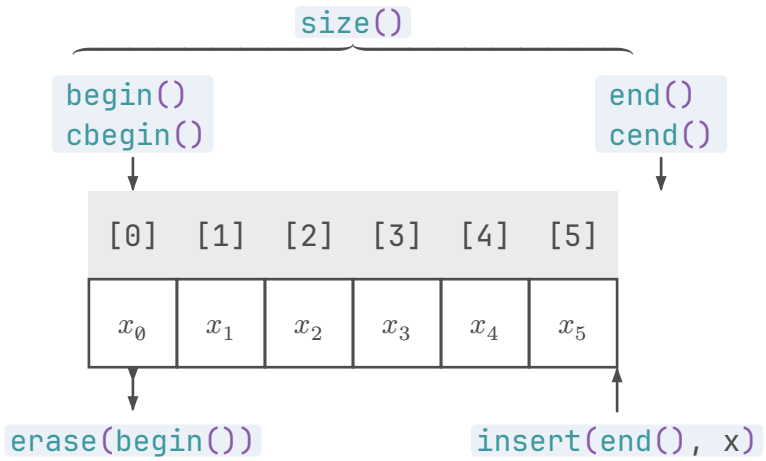
TODO:
W6 39 and the cursedness of 40

9. STL CONTAINERS AND ITERATORS

Sequence Containers	*Associative Containers*	*Hashed Containers (Unordered Associative)*
Elements are accessible in order as they were inserted/created Find in linear time through the algorithm `std::find`	Elements are accessible in sorted order Find as member function in logarithmic time	Elements are accessible in unspecified order Find as member function in constant time
		

All containers have the same/a similar basic interface

Member Function	*Purpose*
`begin()` `end()`	Get iterators for algorithms and iteration in general
`erase(iter)`	Removes the element at position the iterator `iter` points to
`insert(iter, value)`	Inserts `value` at the position the iterator `iter` points to
`size()` `empty()`	Check the size of the container



Containers can be:
– default-constructed `std::vector<int> v{};`
– copy-constructed from another container of the same type `std::vector<int> vv{v};`

– equality compared if they are of the same type (or even lexicographically compared with relational operators) as long as their elements can be compared accordingly `v == vv`
– emptied with `v.clear();`

TODO:
W7 S11

TODO:
diagrams for all containers

9.1. SEQUENCE CONTAINERS

TODO:
W7 S13, explain `std::vector<T>` `std::deque<T>` `std::list<T>` `std::array<T>`

9.1.1. std::array

```
#include <array>

std::array values{1, 1, 3, 6, 9};
```

An array is a fixed-size “container” to which one cannot insert/append. It can be initialized at compile-time. One should use std::array instead of C-style arrays when defining arrays.

9.1.2. std::deque<T>

```
#include <deque>

std::deque<int> q{begin(v), end(v)};
```

`std::deque` is like `std::vector` but with additional, efficient front insertion/removal `push_front()` and `pop_front()`.

TODO:
Special implementation for `bool`: `std::vector<bool>` and `std::deque<bool>`

9.1.3. std::list<T>

```
#include <list>

std::list<int> l(5, 1);
```

It’s basically a doubly-linked list. Efficient insertion in any position `insert(next(begin(), 3), x)`, but lower efficiency in bulk operations. Requires member-function call for `sort()` etc. Only bi-directional iterators - no index access!

9.1.4. std::forward_list<T>

```
#include <forward_list>

std::forward_list<int> l{1, 2, 3, 4, 5, 6};
```

It's basically a singly-linked list, meaning efficient insertion **after** any position, but clumsy with iterator to get "before" position. It only has forward-iterators and it's clumsy to search and remove. Use member-functions not algorithms, or avoid completely, except when there is a specific need. Better use `std::list` or even better `std::vector`

9.2. ADAPTERS

Adapters are no longer a container, due to the deliberate limitation.

9.2.1. std::stack

```cpp
#include <stack>

std::stack<int> s{};
s.push(42);
std::cout << s.top();
s.pop();                // does not return the popped element!
```

This LIFO adapter uses `std::deque` (or `std::vector`, `std::list`) but limits its functionality to stack operations (`push_back()`, `back()` and `pop_back()`). Iteration is thus not possible.

9.2.2. std::queue

```cpp
#include <queue>

std::queue<int> q{};
q.push(42);
std::cout << q.front();
q.pop();
```

This FIFO adapter uses `std::deque` (or `std::list`) and limits its functionality to queue operations (`push_back()` and `pop_front()`). Iteration is thus not possible.

9.2.3. std::priority_queue

```cpp
#include <stack>

std::priority_queue<int> q{};
```

This adapter uses `std::deque` (or `std::vector`) and limits its functionality to stack operations but keeps elements partially sorted as (binary) heap. `top()` element is always the smallest (requires element type to be comparable).

9.3. ASSOCIATIVE CONTAINERS

Associative Containers, also called Sorted Tree Containers, allow searching by content, not by sequence. So you can search by key, access key or key-value pair.

	Key Only	*Key-Value Pair*
Key Unique	`std::set<T>`	`std::map<K, V>`
Multiple Equivalent Keys	`std::multiset<T>`	`std::multimap<K, V>`

9.3.1. std::set<T>

```
#include <set>

std::set<char> vowels{'a', 'e', 'o', 'u', 'i', 'y'}
```

The set stores elements in sorted order (ascending by default, but order can be overwritten by the 2nd template parameter).

TODO:
Iteration walks over the elements in order, keys cannot be modified through iterators. Use member functions for `.find()` and `.count()`
– Tree-search instead of sequential search
– Result of `.count(element)` is either 0 or 1
– Since C++20 there is a `.contains(element)` member

9.3.2. std::map<K, V>

```
#include <map>

std::map<char, size_t> vowels{
  {'a', 3}, {'e', 8}, {'i', 5}, {'o', 4}, {'u', 2}, {'y', 1}
};
```

Map stores key-value pairs in sorted order (Sorted by key in ascending order, order can be overwritten by the 3rd template parameter). Iterators access `std::pair<key, value>`. Use `.first` for key and `.second` for value.

The indexing operator `[]` inserts a new entry automatically if key is not present! The key is the argument of the index operator, values is the default value of the value type. It returns the value by reference, which allows modification.

9.3.3. std::multiset<T> and std::multimap<K, V>

```
#include <set>
#include <map>

std::multiset<char> letters{'a', 'a', 'c', 'c', 'c', 'f'};
std::multimap<int, char> cletters{
  {1, 'a'}, {1, 'c'}, {2, 'a'}, {2, 'c'}, {2, 'f'}
};
```

Multiple equivalent keys allowed, use `equal_range()` or `lower_bound()` / `upper_bound()` member functions/algorithms to find boundaries of equivalent keys.

TODO:
W7 S31,32

9.4. HASHED CONTAINERS

9.4.1. std::unordered_set<T>

```
#include <unordered_set>

std::unordered_set<char> vowels{'a', 'e', 'i', 'o', 'u'};
```

C++11 introduced associative containers using hashing which have more efficient lookup and no sorting. The standard lacks feature for creating your own hash functions, but tere are `std::hash<T>` functor for library types and built-in types provided, esp. `std::string`. DIY programming of hash functions is hard and prone to failure, i.e. it might produce too many collisions → stick to standard types, like `std::string` for keys.

9.4.2. std::unordered_map<K, V>

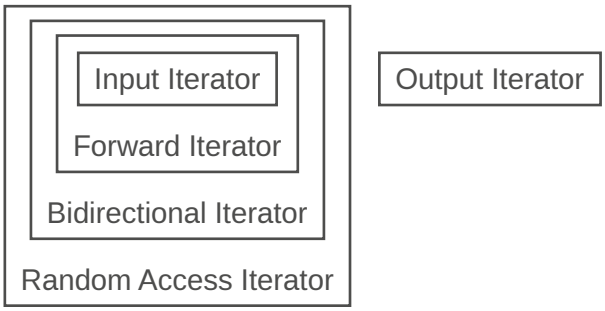
```
#include <unordered_map>

std::unordered_set<std::string, int> words{
  {"theanswer", 42}, {"h4x0r", 1337}, {"nice", 69}
};
```

Usage is almost equivalent to `std::map` except for the lack of ordering.

9.5. ITERATORS

Different containers support iterators of different capabilities. Categories are formed around increasing “power”.
– `std::input_iterator` corresponds to `istream_iterator`’s capabilities
– `std::ostream_iterator` is an `output_iterator`
– `std::vector<T>` provides `random_access` iterators



9.5.1. Categories

Some algorithms only work with powerful iterators, e.g. `std::sort()` requires a pair of random access iterators (it needs to jump forward and backward). Some algorithms can be implemented better with more powerful iterators E.g. `std::advance()` or `std::distance()`.

TODO:
Categories are enforcable with C++20 concepts

***Example** 35:*

```
template<class InputIt, class OutputIt>
auto copy(InputIt first, InputIt last, OutputIt result) → OutputIt;
```

```cpp
template<class RandomAccessIterator>
void sort(RandomAccessIterator first, RandomAccessIterator last);
```

9.5.1.1. Input Iterator

TODO:

Supports reading the "current" element of type `Element`

It allows for one-pass input algorithms, cannot step backwards.

It models the `std::istream_iterator` and `std::istream`

It can be compared with `==` and `!=` to other iterator objects of the same type: `It`

It can be copied, after increment (calling `++`) all other copies are invalid! `*it++` is allowed explicitly (by the standard)

```cpp
struct input_iterator_tag{};

auto operator* () -> Element;
auto operator++() -> It&;
auto operator++(int) -> It;
auto operator==(It const&) -> bool;
auto operator!=(It const&) -> bool;
auto operator= (It const&) -> It&;
It(It const&); //copy ctor
```

9.5.1.2. Forward Iterator

TODO:

It can do whatever an input iterator can, plus changing the "current" element (of type `Element`), unless the container or its elements are `const`.

It still allows only for one-pass input algorithms, cannot step backwards, but can keep iterator copy around for later reference.

It models the `std::forward_list` iterators.

```cpp
struct forward_iterator_tag{};

// all from input iterator
```

9.5.1.3. Bidirectional Iterator

TODO:

It can do whatever a forward iterator can, plus going backwards. It allows for forward-backward-pass algorithms and models the `std::set` iterators.

```cpp
struct
bidirectional_iterator_tag{};

// all from forward iterator plus
auto operator--() -> It&;
auto operator--(int) -> It;
```

9.5.1.4. Random Access Iterator

TODO:

It can do whatever a bidirectional iterator can, plus:

```cpp
struct
random_access_iterator_tag{};
```

– Directly access element at index (offset to current position): distance can be positive or negative
– Go n steps forward or backward
– “Subtract” two iterators to get the distance
– Compare with relational operators (`<`, `⩽`, `>`, `⩾`)

It allows random access in algorithms and models the `std::vector` iterators.

```
// all from bidirectional iterator
plus
auto operator[](distance) →
Element&;
auto operator+(distance) → It;
auto operator+=(distance) → It&;
auto operator-(distance) → It;
auto operator-=(distance) → It&;
auto operator-(It const &) →
distance;
//relational operators, like <
```

9.5.1.5. Output Iterator

TODO:

It can write value to current element, but only once (`*it = value`), then increment is required. Itys modeled after `std::ostream_iterator`. Most other iterators can also act as output iterators, unless the underlying container is const, except for associative containers, they allow only read-only iteration. No comparison and end to an out range is not queryable.

```
struct output_iterator_tag{};

auto operator*() → Element&;
auto operator++() → It&;
auto operator++(int) → It;
```

9.5.2. Functions

9.5.2.1. std::distance

```
#include <iterator>;

std::distance(start, goal);
```

It counts the number of “hops” iterator start must make until it reaches goal. Efficient for random access iterators, for other iterators the algorithm has to hop to the element one-by-one. This implies that goal has to be “after” start. I.e. reachable by an arbitrary number of `++` calls.

9.5.2.2. std::advance

```
#include <iterator>;

std::advance(iter, n);
```

It lets `iter` “hop” `n` times. Efficient for random access iterators, for other iterators, the algorithm has to loop. It also allows negative `n` for bidirectional iterators.

9.5.2.3. std::next

```
#include <iterator>;

std::next(iter);
```

It has a default step of size 1, but can be specified. It makes a copy of the argument, the argument can also be a temporary.

`std::advance` on the other hand requires a step, modifies the argument iterator and returns void.

***Example** 36:*

```
auto main() → int {
  std::vector primes{2, 3, 5, 7, 11, 13};

  auto current = std::begin(primes);
  auto afterNext = std::next(current);
  std::cout << "cur: " << *current << " after: " << *afterNext << '\n';

  std::advance(current, 1);
  std::cout << "cur: " << *current << " after: " << *afterNext << '\n';
}
```

9.5.3. A word on mutability

Declaring an iterator `const` would not allow modifying the iterator object (E.g. you cannot call `++`). `cbegin()` and `cend()` return `const_iterators`: This does not imply the iterator to be `const`, the elements the iterator walks over are `const`.

BIBLIOGRAPHY

[1] “C++ Mascot.” Accessed: Sep. 12, 2026. [Online]. Available: https://media.licdn.com/dms/image/v2/D4D22AQFwZZ5wmpZmnw/feedshare-shrink_800/feedshare-shrink_800/0/1724256362408?e=2147483647&v=beta&t=OXxfC_GlaJFqlUgO03e-9B59Zpzu4-LYG90lM8k4UFE

TODO:

	`Date d;`	`Date d{};`	`Date d{1,2,3};`
`struct Date { int year, month, day; };`			
`class Date { int year, month, day; };`			
`class Date { int year{4}, month{5}, day{6}; };`			
`class Date { int year{4}, month{5}, day{6}; }; Date(int year, int month, int day) : year{year},month{month},day{day} { };`			